

Clubuntu Deploy Pipeline

How a commit made on a laptop ends up running as containers on a Linux cloud server it never touches directly — and where that design sits among the deployment strategies used in industry.

scope: ManPage/ → GitHub → clubuntu (Tailscale node) 2026-08-16

🔗 Deployment strategies, an introduction

A deployment strategy answers one question: how does code that changed on a developer's machine end up running, reliably, as the thing people or other tools actually talk to? Nearly everything in production use today is built from the same handful of ingredients, combined differently depending on scale and risk tolerance.

Ad-hoc / manual

SSH into the server, pull the latest code, restart the process. Fastest to set up, no artifact history, no rollback beyond "SSH back in and pull again," and every deploy depends on whatever state that server happens to already be in. Fine for a single personal project; a liability past that.

Immutable artifact deployment

Build a versioned, self-contained artifact once — a container image is the dominant form today — store it somewhere addressable, and deploy by pointing the target machine at a specific version of that artifact rather than at a moving source tree. The artifact never changes after it's built: `v1.4.0` means the exact same bytes everywhere it runs. This is the foundation everything else here builds on, and it's what a container registry is for.

CI/CD automation

The build-and-push half of the above, done by a machine on a trigger instead of a person typing commands. Removes "it worked on my laptop, the build was different on the server"

as a category of bug, and gives every deploy a paper trail — which commit, which pipeline run, which artifact.

GitOps

A further step some teams take: instead of a pipeline *pushing* a deploy to a target, an agent running *at* the target continuously watches a declared desired state — usually a git repo — and pulls itself into sync. Argo CD and Flux do this for Kubernetes; Watchtower does a lighter version for plain Docker Compose. It buys drift correction (a manual change on the server gets quietly reverted) at the cost of another moving part to operate.

Release patterns

Once new versions deploy reliably, the next question is how existing traffic experiences the switch — all at once, gradually, via a full parallel environment, or to a small slice of traffic first. These solve a problem this project doesn't have yet — multiple live replicas of the same service — but they're the industry-standard vocabulary for it, covered in §05.

The design in this document sits at **immutable artifacts + CI/CD automation**, with a deliberate manual gate — a git tag — before anything deploys, rather than full GitOps or deploy-on-every-commit. That's a specific, considered point on the spectrum above, not a lesser version of it.

01 Goal and constraints

The dev machine is authoritative — changes are made there. The deployed containers on clubuntu should update in response, but three things stay fixed:

- ▶ The private registry stays exactly as private as it is today — reachable only over Tailscale, never the public internet.
- ▶ Deploys happen deliberately, on a tag — not automatically on every push. A half-finished commit should never take down what's running.
- ▶ The source tree is not the thing that gets shipped. Only images are.

02 Components: the machines and networks involved

Four kinds of thing are involved, and it's worth being precise about what each one is and isn't responsible for.

Dev laptop(s)

Where code is written and committed. Nothing about this changes — the same `git push` already run today is the only interaction point. More than one laptop can push; the pipeline doesn't care which node a commit came from, only that it landed on GitHub.

Tailscale network

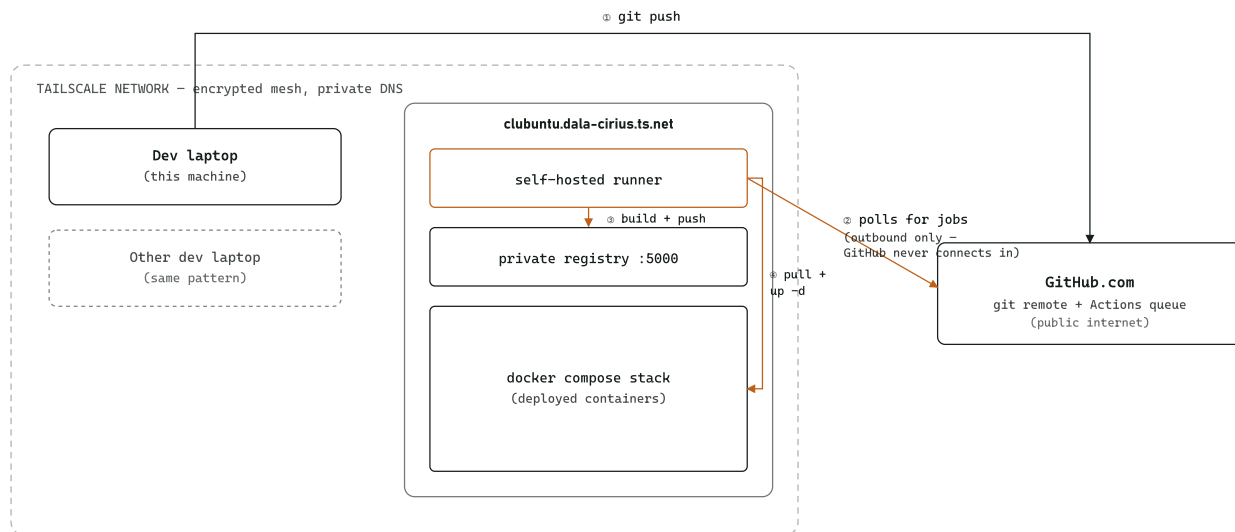
A private, encrypted mesh (WireGuard underneath) connecting every enrolled device — laptops and clubuntu alike — under stable private DNS names (`clubuntu.dala-cirius.ts.net`) regardless of where each device physically is or what NAT it sits behind. Nothing on this network is reachable from the public internet unless deliberately exposed; the registry stays exactly this private throughout everything below. It's also unauthenticated by design — access is controlled entirely by tailnet membership, not a login, so nothing in the pipeline below needs a registry credential at all.

clubuntu — Linux Cloud Server

One machine inside the tailnet doing three jobs at once: hosting the private registry, running the CI runner, and running the deployed stack itself. All three talk to each other over localhost — nothing between them ever leaves the machine.

GitHub

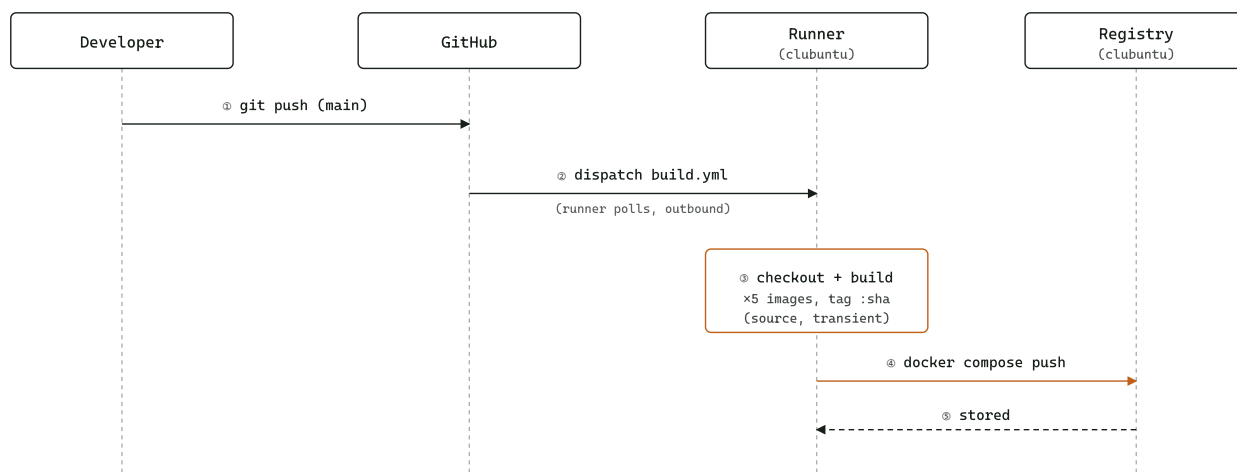
Lives entirely outside the tailnet, on the public internet. It only ever does two things here: receive pushes, and hand a queued job to whichever runner asks for one. It never initiates a connection into the tailnet — the runner reaches out to GitHub, not the other way around. That direction is what makes this whole design not require opening a port or exposing anything new.



Every hop inside the dashed boundary is localhost or the tailnet itself. Only step ① (a laptop pushing to GitHub) and step ② (the runner polling GitHub) ever cross into the public internet — and both are outbound, initiated from inside the tailnet. Nothing about this design opens a port.

03 Build sequence — every push to main

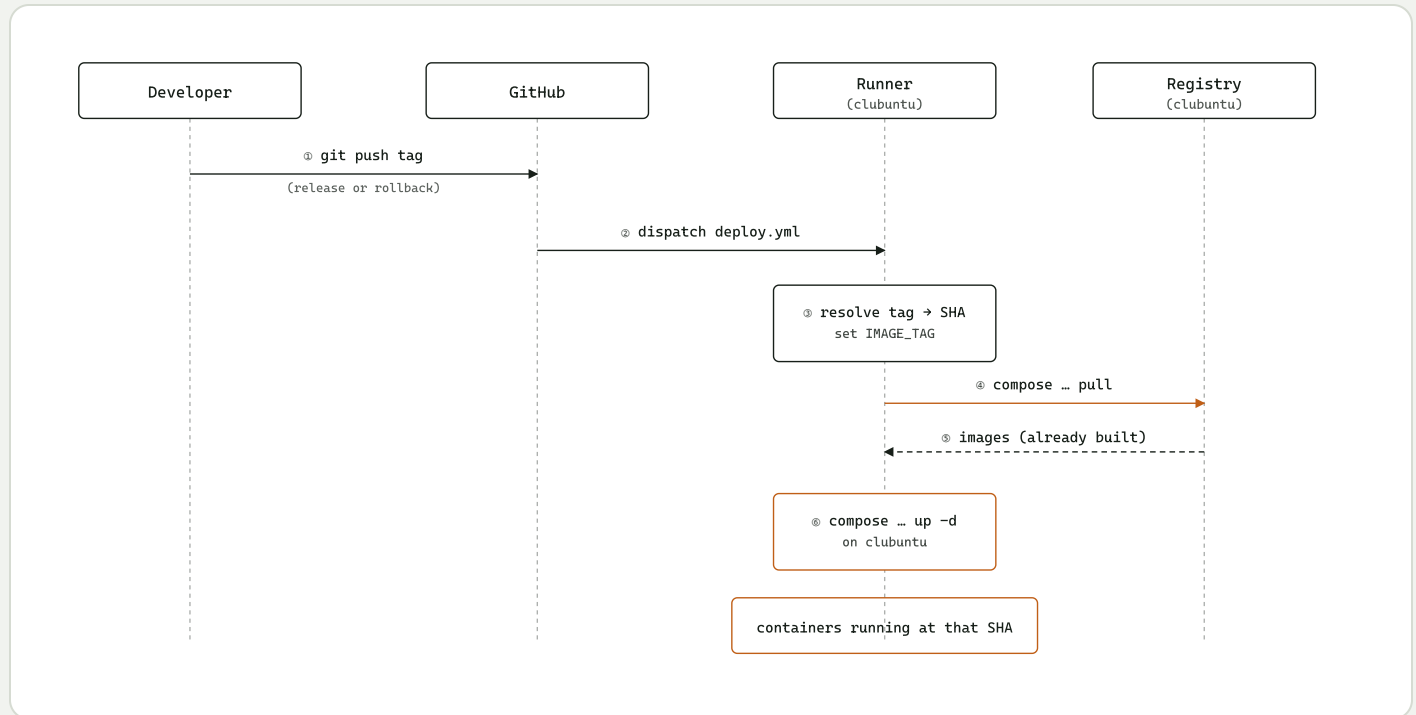
Builds are cheap and constant. Every push produces five freshly built, SHA-tagged images and stocks the registry with them — and stops there. Nothing gets deployed by this alone.



This sequence never touches the running stack — it only stocks the registry. A commit can be built a dozen times without a single container on clubuntu ever restarting.

04 Deploy sequence — on tag, and rollback for free

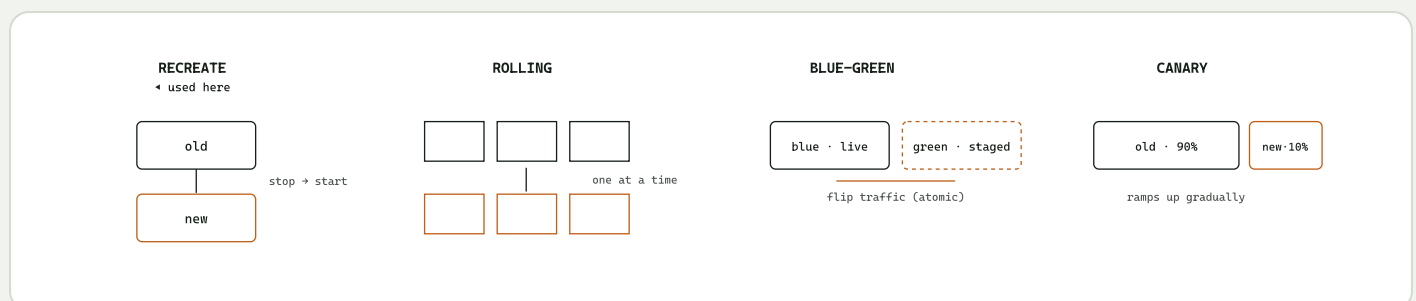
A tag and a rollback run through the identical sequence below. The only thing that differs is which commit the tag happens to point at — the entire payoff of tagging a SHA that was already built in §03, rather than building on deploy.



Rollback isn't a special case drawn separately — it's this exact sequence, entered with an older tag. Because every push in §03 already built and pushed images by SHA, step ④ pulls something that's already sitting in the registry; step ⑥ never rebuilds anything.

05 Where this sits among industry release patterns

§03–04 cover *getting a new image onto the server*. A separate, later question — once a service has more than one running instance — is *how live traffic experiences the switch*. Four patterns cover nearly everything used in production today:



Neutral boxes are the old version, amber the new one. Recreate is the only pattern that doesn't need more than one running instance of a service — which is exactly the situation every service in this stack is in today.

PATTERN	HOW IT WORKS	DOWNTIME	REQUIRES	USED HERE?
Recreate	Stop the old instance, start the new one	Brief (seconds)	Nothing extra	Yes — this pipeline
Rolling	Replace instances one at a time behind a stable set	None, with 2+ replicas	2+ replicas per service	No — every service here is a single instance
Blue-Green	Run a full second environment, switch traffic atomically	None	A router able to flip traffic atomically	No — nothing sits in front of this stack today
Canary	Send a small % of traffic to the new version first, expand gradually	None	Traffic-splitting + metrics to watch	No — solo-operator internal tool; relevant if this ever serves outside users

06 What has to change in this repo

One prerequisite (this becomes a real git repo), four file-level changes, one machine-level setup step. None of these exist yet.

DOCKER-COMPOSE.YML — ADD IMAGE: ALONGSIDE EVERY BUILD:

ONE REPRESENTATIVE SERVICE

```
coder-commands-mcp:
  build: ./coder-commands-mcp
  image: clubuntu.dala-cirius.ts.net:5000/coder-commands-mcp:${IMAGE_TAG:-dev}
  container_name: coder-commands-mcp
  ...
```

With this in place, `docker compose build` / `push` / `pull` all work against the registry, unchanged from today's manual flow — `IMAGE_TAG` just needs to be set in the environment (CI sets it; a local build without it falls back to `:dev`).

NEW: DOCKER-COMPOSE.DEPLOY.YML — IMAGE-ONLY, NO BUILD:

ABRIDGED

```

name: codercommands

services:
  postgres:
    image: postgres:17
    ...
  coder-commands-mcp:
    image: clubuntu.dala-cirius.ts.net:5000/coder-commands-mcp:${IMAGE_TAG}
    ...
  gateway:
    image: clubuntu.dala-cirius.ts.net:5000/gateway:${IMAGE_TAG}
    ...
# docker-mcp, postgres-mcp, sync follow the same shape

```

This is the file that actually runs on clubuntu. It never needs the Dockerfiles or source present — every service resolves to an already-built image.

NEW: `.GITHUB/WORKFLOWS/BUILD.YML`

```

name: build
on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: [self-hosted, clubuntu]
    steps:
      - uses: actions/checkout@v4
      - run: |
          export IMAGE_TAG=${GITHUB_SHA::7}
          docker compose build
          docker compose push

```

NEW: `.GITHUB/WORKFLOWS/DEPLOY.YML`

```

name: deploy
on:
  push:
    tags: ['v*']

jobs:
  deploy:
    runs-on: [self-hosted, clubuntu]
    steps:

```

```

- uses: actions/checkout@v4
- run: |
    SHA=$(git rev-list -n 1 ${GITHUB_REF_NAME})
    echo "IMAGE_TAG=${SHA::7}" >> $GITHUB_ENV
- run: |
    # the checkout above is a fresh, disposable directory --
    # .env lives only in the persistent deploy directory below,
    # so the compose file moves to where .env already is, not
    # the other way around.
    cp docker-compose.deploy.yml ~/codercommands/docker-compose.deploy.yml
- run: |
    cd ~/codercommands
    docker compose -f docker-compose.deploy.yml pull
    docker compose -f docker-compose.deploy.yml up -d

```

`~/codercommands` above is a placeholder for whatever directory clubuntu already deploys from today — the one with a working `.env` in it. Swap it for the real path once confirmed; nothing else about this step changes.

07 Locked decisions, and what's outside what I can do

Three things settled this round, precise enough to build from:

- ▶ Registry is unauthenticated — no `docker login`, no registry credential anywhere in the workflows.
- ▶ Deploy reuses clubuntu's existing deploy directory (the one with a working `.env` already in it) rather than a fresh location — reflected in §06's `deploy.yml` above.
- ▶ The GitHub repo doesn't exist yet and needs creating, private.

Splitting what happens next by who can actually do it:

- ▶ **Needs the GitHub web UI (or installing/authenticating `gh`, neither present on this machine):** creating the empty private repo itself. One-time, a few clicks.
- ▶ **I can do once that empty repo exists:** `git init`, the initial commit, `git remote add origin`, and `git push` — plain `git` is already installed here. The push may pause for a one-time browser sign-in if this machine hasn't authenticated to GitHub before.
- ▶ **Needs GitHub's UI, then clubuntu itself:** registering the self-hosted runner — a one-time registration token from repo Settings → Actions → Runners → New self-hosted

runner, then running the setup script *on clubuntu*. I don't have direct access to that machine from this session; happy to write out the exact commands to paste there once the repo exists.

- ▶ **Needs confirming on clubuntu:** the real path to swap in for the `~/codercommands` placeholder in §06's `deploy.yml`.

ManPage/docker-compose.yml · ManPage/docker-compose.deploy.yml (new) ·
.github/workflows/build.yml (new) · .github/workflows/deploy.yml (new)